Generative Score-Based Models

MATH*4600 Final Report

Vinay Joshy*

April 10, 2025

Abstract

This paper investigates generative score-based models, a promising approach to generative modeling that directly estimates the gradients of data distribution's log probability density. We compare two primary score estimation techniques: sliced score matching (SSM) and denoising score matching (DSM), analyzing their performance across different data complexities. SSM reduces computational complexity by projecting high-dimensional scores onto random directions, while DSM leverages a denoising perspective that avoids computing Hessians. Experiments were conducted on both synthetic Gaussian Mixture Models, where analytical scores enabled direct accuracy assessment, and the FashionMNIST dataset, where performance was evaluated through generated sample quality. Our results demonstrate that SSM provides more accurate score approximations than DSM, particularly in critical low-density regions between mixture components, leading to higher quality generated samples with clearer structures and better preservation of characteristic features. Despite differences in score estimation accuracy, both methods successfully recovered target distributions, suggesting generative performance remains robust to certain types of approximation errors. These findings contribute to the growing understanding of score-based generative models, which combine strengths of explicit density modeling and implicit sampling-based approaches while avoiding challenges associated with density normalization.

1 Introduction

Generative models have exploded in the past three years, with over a billion people actively using some sort of generative technology in their daily lives[1]. Two of the most popular generative modeling techniques include likelihood-based models and implicit generative models [2, 3]. Likelihood-based models learn the probability density of distribution directly by estimating the maximum likelihood. Examples include auto-regressive models [4], variational

*By submitting this document for grading, we confirm that this work is our own and that we have adhered to the University of Guelph's Academic Integrity Policies as outlined in the Undergraduate Calendar. We recognize that any violation of these policies, whether intentional or not, may constitute academic misconduct to which we may be subject to an Academic Misconduct Investigation and penalized accordingly if found guilty.

auto-encoders [5], normalizing flow models [6], etc. Implicit generative models, on the other hand, implicitly represent the probability distribution via a model of its sampling process. Generative adversarial networks are a well-known example of this.

Score-based models provides a novel approach to generative modeling by directly estimating the gradients of the data distribution's log probability density [7]. This technique has shown a marked improvement over other generative modeling techniques [7, 8]. Score estimation on raw data can still be challenging due to the sparsity of data in high-dimensional spaces, particularly in low-density regions where accurate estimation is crucial for sampling procedures.

In this paper, we investigate and compare two primary score estimation techniques: sliced score matching (SSM) and denoising score matching (DSM). These approaches offer different computational and theoretical advantages for estimating score functions. SSM reduces computational complexity by projecting high-dimensional scores onto random directions, while DSM leverages a denoising perspective that bypasses the need to compute Hessians entirely. We examine how these methods perform across different data dimensionality and complexity levels.

The paper is organized as follows: First, we provide background on score-based generative models, including the theory of score matching, sampling with Langevin dynamics, use of stochastic differential equations in generative models and different score estimation procedures. We then detail our methods, describing experimental setups for both synthetic Gaussian Mixture Models and the FashionMNIST dataset. The results section presents a comparative analysis of SSM and DSM performance across these datasets, followed by a discussion of our findings and their implications. We conclude by outlining limitations of our approach and suggesting directions for future research.

2 Background

The goal of most generative models is to represent probability distributions that approximate the data distribution well. Given i.i.d samples $\{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_N\} \subset \mathbb{R}^D$ from a data distribution, $p_d(\mathbf{x})$, we must learn some unnormalized density $\tilde{p}_m(\mathbf{x}; \theta)$ where $\theta \in \Theta$. The normalized density determined by the model is defined as:

$$p_m(\mathbf{x}; \theta) = \frac{\tilde{p}_m(\mathbf{x}; \theta)}{Z_\theta}, \quad Z_\theta = \int \tilde{p}_m(\mathbf{x}; \theta) \mathbf{dx}$$

where the partition function of the model, Z_θ, is often intractable. Hyvarinen [9] showed that the Fisher divergence between p_d and $p_m(., \theta)$ is minimized via score-matching. Fisher divergence is defined as:

$$L(\theta) \triangleq \frac{1}{2} \mathbb{E}_{p_d}[||\mathbf{s}_m(\mathbf{x}; \theta) - \mathbf{s}_d(\mathbf{x})||_2^2] \tag{1}$$

where $\mathbf{s}_m(\mathbf{x}; \theta) \triangleq \nabla_\mathbf{x} \log\ p_m(\mathbf{x}; \theta)$ and $\mathbf{s}_d(\mathbf{x}) \triangleq \nabla_\mathbf{x} \log\ p_d(\mathbf{x})$ represent the score functions of p_m and p_d respectively. Note that the score function of the normalized density is independent of the partition function, Z_θ. This is because Z_θ is a normalization constant that integrates over all possible values of $\mathbf{x}$ making it only dependent on θ (See Appendix A).

To use Equation 1 we need access to the score function of the data which is rarely available. Instead Hyvarinen [9] shows that the Fisher divergence can be written as $L(\theta) = J(\theta) + C$ (See Appendix B) where

$$J(\theta) \triangleq \mathbb{E}_{p_d}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x};\theta)\|_2^2\right], \tag{2}$$

C is a constant independent of θ and tr(.) denotes the trace of a matrix. This gives us an unbiased estimator:

$$\hat{J}(\theta;\underset{\sim}{\mathbf{x}}) = \frac{1}{N}\sum_{i=1}^{N}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x}_i;\theta)) + \frac{1}{2}\|\mathbf{s}_m(\mathbf{x}_i;\theta)\|_2^2\right] \tag{3}$$

where $\underset{\sim}{\mathbf{x}}$ denotes the collection of i.i.d samples $\{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_N\} \subset \mathbb{R}^D$ from p_d and $\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x}_i;\theta))$ denotes the Hessian of $\log \tilde{p}_m(\mathbf{x};\theta)$ evaluated at $\mathbf{x}_i$.

2.1 Summary of Song and Ermon [7]: Generative Modeling

Score-based generative models offer a novel approach to generative modeling by directly estimating the gradients of a data distribution's log probability density (the score function). Song and Ermon [7] introduced a framework that addresses two challenges in score-based methods: undefined scores on low-dimensional data manifolds and inaccurate estimation in low-density regions. Their approach perturbs data with multiple levels of Gaussian noise and trains a single conditional score network to estimate scores at all noise levels. For sampling, they developed annealed Langevin dynamics, which gradually decreases noise levels during the sampling process. This technique achieved state-of-the-art results on complex image datasets like CIFAR-10, demonstrating that score-based methods can produce samples comparable in quality while avoiding adversarial training and specialized architectures. This framework is the main inspiration for this paper and the following subsections are summarized from Song and Ermon [7].

2.1.1 Sampling with Langevin Dynamics

Once we have trained a score-based model $\mathbf{s}_m(\mathbf{x};\theta)$ to approximate the score function $\mathbf{s}_d(\mathbf{x})$, we need a method to generate samples from the corresponding probability distribution. Langevin dynamics provides a sampling mechanism that requires only the score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ of a target distribution [10].

Given a step size $\epsilon > 0$ and an initial value $\mathbf{x}_0 \sim \pi(\mathbf{x})$ from some prior distribution π, the Langevin method iteratively computes:

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \frac{\epsilon}{2}\nabla_{\mathbf{x}} \log p(\mathbf{x}_{t-1}) + \sqrt{\epsilon}\mathbf{z}_t, \tag{4}$$

where $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$. Under suitable regularity conditions, as $\epsilon \to 0$ and the number of iterations $T \to \infty$, the distribution of $\mathbf{x}_T$ converges to the target distribution $p(\mathbf{x})$.

For finite values of $\epsilon > 0$ and $T < \infty$, the samples from Langevin dynamics represent an approximation to $p(\mathbf{x})$. While a Metropolis-Hastings correction step would be theoretically

necessary to address this approximation error, in practice it can often be omitted when ϵ is sufficiently small and T is large [7].

Importantly, Langevin dynamics sampling only requires the score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. Therefore, to sample from the data distribution $p_d(\mathbf{x})$, we first train a score-based model $\mathbf{s}_m(\mathbf{x}; \theta)$ such that $\mathbf{s}_m(\mathbf{x}; \theta) \approx \nabla_{\mathbf{x}} \log p_d(\mathbf{x})$, and then generate samples using Langevin dynamics with $\mathbf{s}_m(\mathbf{x}; \theta)$.

2.1.2 Challenges with Score Estimation

A major challenge with score-based models is that the estimated score functions are often inaccurate in low density regions, where few data points are available for computing the score matching objective [7]. This inaccuracy stems directly from how score matching minimizes the Fisher divergence:

$$\mathbb{E}_{p_d}[||\nabla_{\mathbf{x}} \log p_d(\mathbf{x}) - \mathbf{s}_m(\mathbf{x}; \theta)||_2^2] = \int p_d(\mathbf{x})||\nabla_{\mathbf{x}} \log p_d(\mathbf{x}) - \mathbf{s}_m(\mathbf{x}; \theta)||_2^2 d\mathbf{x}$$

Since the ℓ_2 differences between the true data score function and score-based model are weighted by $p_d(\mathbf{x})$, errors are largely ignored in low density regions where $p_d(\mathbf{x})$ is small. This behavior leads to subpar generative results, particularly when using Langevin dynamics for sampling [7].

When sampling with Langevin dynamics, our initial samples are highly likely to begin in low density regions when data reside in a high-dimensional space. Therefore, having an inaccurate score-based model will derail Langevin dynamics from the very beginning of the procedure, preventing it from generating high-quality samples that are representative of the data distribution.

2.1.3 Noise Perturbation Procedure

To overcome the challenge of inaccurate score estimation in regions of sparse data density, we introduce controlled noise to the original data points. This effectively populates the low density regions of the data space, enabling more reliable score estimation throughout the entire domain. The perturbation strategy involves adding Gaussian noise to data samples at increasing scales [7, 8]. Specifically, a sequence of L progressively increasing standard deviations $\sigma_1 < \sigma_2 < \cdots < \sigma_L$ for Gaussian noise perturbation (See Figure 1). For each noise scale σ_i, we create a perturbed distribution:

$$p_{\sigma_i}(\mathbf{x}) = \int p_d(\mathbf{y}) \mathcal{N}(\mathbf{x}; \mathbf{y}, \sigma_i^2 \mathbf{I}) d\mathbf{y} \tag{5}$$

Samples are generated from these perturbed distributions by drawing $\mathbf{x} \sim p_d(\mathbf{x})$ and adding noise via $\mathbf{x} + \sigma_i \mathbf{z}$, where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

For each noise-perturbed distribution, a specialized model is trained to estimate its score function. This takes the form of a Noise Conditional Score-Based Model $\mathbf{s}_\theta(\mathbf{x}, i)$, which we train using score matching techniques to approximate $\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x})$ across all noise levels $i = 1, 2, \cdots, L$. This conditioning on noise levels allows a single model to effectively capture score information across multiple perturbation scales.

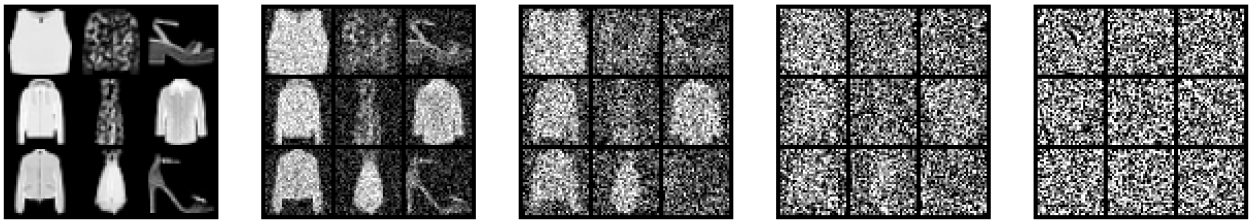


Figure 1: Gaussian noise perturbation procedure on subset of FashionMNIST dataset

2.2 Summary of Song et al. [8]: Generative Modeling via SDEs

Score-based generative models can be elegantly unified and extended through stochastic differential equations (SDEs). Song et al. [8] introduced a framework where data distributions are gradually transformed into noise through a continuous-time diffusion process governed by forward SDEs. To generate samples, they then solve the corresponding reverse-time SDE, which depends on the score function $\nabla \mathbf{x} \quad \log p_t(\mathbf{x})$. This approach generalizes previous score-based and diffusion models by interpreting them as discretizations of specific SDEs. The framework enables new sampling algorithms, including Predictor-Corrector methods that combine numerical SDE solvers with score-based MCMC techniques, and probability flow ODEs that allow exact likelihood computation and efficient sampling. Additionally, the SDE formulation enables controllable generation through conditional reverse-time SDEs, facilitating applications like class-conditional generation, image inpainting, and colorization without model retraining Song et al. [8]. The core ideas applied in the following subsections were summarized from this SDE framework, particularly the formulation of score-based diffusion as a continuous process and sample generation from their reverse time analogs.

2.2.1 Diffusion Process

Utilizing an increasing number of noise scales improves the accuracy of score-based generative models by providing better coverage across the data domain [7]. As the number of scales approaches infinity, this discrete noise perturbation procedure converges to a continuous-time stochastic process known as a diffusion process [8]. A diffusion process $\{\mathbf{x}(t)\}_{t=0}^{T}$ is indexed by a continuous time variable $t \in [0, T]$ and represents a gradual transformation from the data distribution to a simple noise distribution. Specifically, the process starts from the data distribution $\mathbf{x}(0) \sim p_d$ and progressively adds noise until reaching some tractable distribution $\mathbf{x}(T) \sim p_T$, typically a standard Gaussian. This provides a complete continuum of intermediate distributions rather than discrete snapshots [8].

Such a diffusion process can be mathematically modeled using a stochastic differential equation (SDE) of the form:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}, \tag{6}$$

where $\mathbf{f}(., t) : \mathbb{R}^d \to \mathbb{R}^d$ is the drift coefficient controlling the deterministic part of the transformation, $g(t) \in \mathbb{R}$ is the diffusion coefficient determining the noise magnitude, and $d\mathbf{w}$ represents infinitesimal increments of a standard Brownian motion (See Figure 2).

We now train a Time-Dependent Score-Based Model $s_\theta(\mathbf{x}, t)$ to approximate the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ at every time point t. This represents a natural extension of the discrete

case, where $s_\theta(\mathbf{x}, t)$ corresponds to $\mathbf{s}_\theta(\mathbf{x}, i)$ in the finite noise scale approach, t corresponds to noise level $i \in \{1, 2, ..., L\}$, and $p_t(\mathbf{x})$ corresponds to the noise-perturbed distribution $p_{\sigma_i}(\mathbf{x})$ in Equation 5.

2.2.2 Sample Generation using Reverse SDEs

A remarkable property of diffusion processes is that they can be reversed to convert noise back into structured data [8]. Any forward SDE has a corresponding reverse-time SDE with a closed form defined as:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\bar{\mathbf{w}}, \tag{7}$$

where dt is the negative infinitesimal time step and $d\bar{\mathbf{w}}$ denotes a standard Wiener process in reverse time [11].

The reverse SDE depends on the score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ of the marginal distribution at time t. By replacing this with our trained score-based model $s_\theta(\mathbf{x}, t)$, we obtain:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)s_\theta(\mathbf{x}, t)]dt + g(t)d\bar{\mathbf{w}}, \tag{8}$$

To generate samples, we first draw $\mathbf{x}(T) \sim p_T$ from the tractable noise distribution, then numerically solve the reverse SDE in Equation 8 backward in time from $t = T$ to $t = 0$. The resulting $\mathbf{x}(0)$ is an approximate sample from the data distribution p_d. This completes the generative process: noise is transformed into structured data following the reverse path of the forward diffusion.

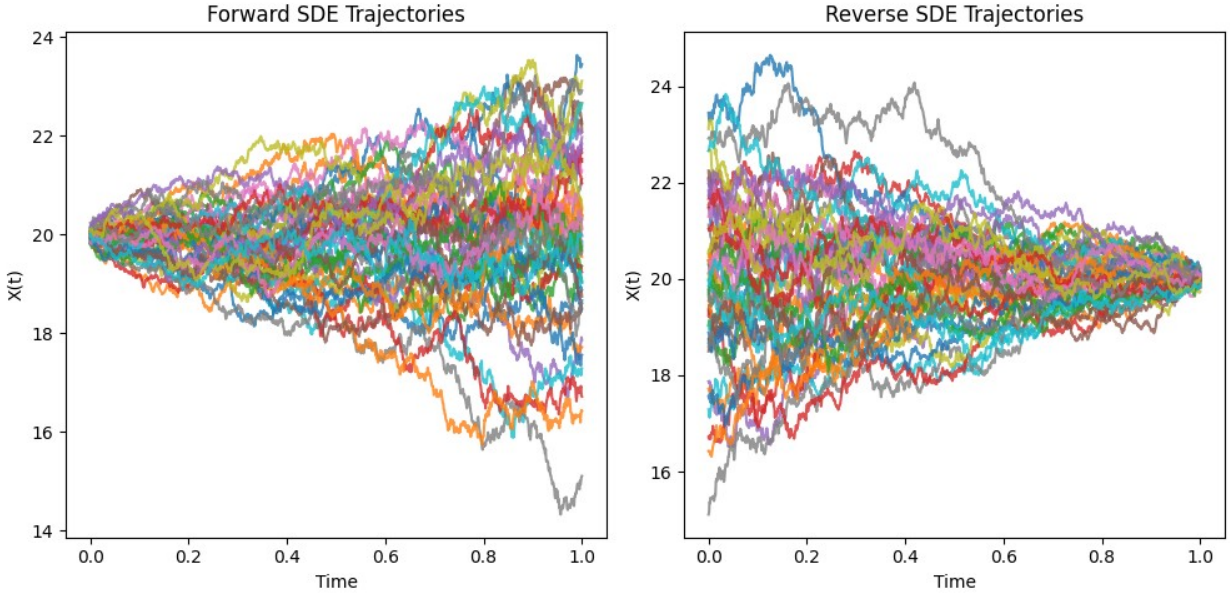


Figure 2: Solutions of forward SDE (left) and reverse-time SDE (right). SDE used is $d\mathbf{x} = e^t dt$, where $t \in [0, 1]$

2.3 Summary of Song et al. [12]: Sliced Score Matching (SSM)

Computing the trace of the Hessian of the score function, $\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta))$, in the score-matching objective (Equation 3) is computationally challenging as it requires D times more backward passes than computing the gradient $s_m = \nabla_{\mathbf{x}} \log\ \tilde{p}_m(\mathbf{x};\theta)$ [12].

Song et al. [12] propose Sliced Score Matching (SSM), a computationally efficient variant of score matching that scales to deep neural networks and high-dimensional data. Projecting high-dimensional scores onto random directions reduces the computational complexity without compromising estimation accuracy. Theoretically, they prove SSM is consistent and asymptotically normal under standard regularity conditions.

Projecting $s_d(\mathbf{x})$ and $s_m(\mathbf{x};\theta)$ onto some random direction $\mathbf{v}$ and comparing their average difference along that random direction, the Fisher divergence becomes:

$$L(\theta; p_{\mathbf{v}}) \triangleq \frac{1}{2}\mathbb{E}_{p_{\mathbf{v}}}\mathbb{E}_{p_d}[(\mathbf{v}^{\mathsf{T}}\mathbf{s}_m(\mathbf{x};\theta) - \mathbf{v}^{\mathsf{T}}\mathbf{s}_d(\mathbf{x}))^2] \tag{9}$$

where $\mathbf{v} \sim p_{\mathbf{v}}$ (typically a multivariate standard normal or Rademacher distribution). Applying integration by parts trick from Hyvarinen [9], the score matching objective becomes:

$$J(\theta; p_{\mathbf{v}}) \triangleq \mathbb{E}_{p_{\mathbf{v}}}\mathbb{E}_{p_d}\left[\mathbf{v}^{\mathsf{T}}\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)\mathbf{v} + \frac{1}{2}(\mathbf{v}^{\mathsf{T}}\mathbf{s}_m(\mathbf{x};\theta))^2\right] \tag{10}$$

In practice, given a dataset $\underset{\sim}{\mathbf{x}}$, we draw M projection vectors independently for each data point $\mathbf{x}_i$ from $p_{\mathbf{v}}$. The collection of all such vectors $\{\mathbf{v}_{ij}\}_{1\leq i\leq N, 1\leq j\leq M}$ is abbreviated as $\underset{\sim}{\mathbf{v}}$. We then use the following unbiased estimator of $J(\theta; p_{\mathbf{v}})$:

$$\hat{J}(\theta; \underset{\sim}{\mathbf{x}}, \underset{\sim}{\mathbf{v}}) \triangleq \frac{1}{N}\frac{1}{M}\sum_{i=1}^{N}\sum_{j=1}^{M}\left[\mathbf{v}_{ij}^{\top}\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x}_i;\theta)\mathbf{v}_{ij} + \frac{1}{2}(\mathbf{v}_{ij}^{\top}\mathbf{s}_m(\mathbf{x}_i;\theta))^2\right] \tag{11}$$

Computing the second derivative term $\mathbf{v}^{\mathsf{T}}\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)\mathbf{v}$ is much more efficient than computing $\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta))$, requiring only two gradient operations for a single $\mathbf{v}$ in modern auto-differentiation frameworks. Experimentally Song et al. [12] showed that SSM outperforms other scalable score matching variants for density estimation and score estimation tasks, including training deep energy models and Wasserstein Auto-Encoders.

2.4 Denoising Score Matching (DSM)

While the sliced score matching approach helps mitigate computational challenges of traditional score matching, we can consider an alternative approach. This method, known as Denoising Score Matching (DSM) [13], works with pairs of clean and corrupted examples $(\mathbf{x}, \tilde{\mathbf{x}})$ where $\tilde{\mathbf{x}} = \mathbf{x} + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2\mathbf{I})$. For a joint density $p_{\tilde{\mathbf{x}},\mathbf{x}}(\tilde{\mathbf{x}}, \mathbf{x}) = p_{\tilde{\mathbf{x}}|\mathbf{x}}(\tilde{\mathbf{x}}|\mathbf{x})p_d(\mathbf{x})$, where $p_d(\mathbf{x})$ is the data distribution and $p_{\tilde{\mathbf{x}}|\mathbf{x}}(\tilde{\mathbf{x}}|\mathbf{x})$ is a noise model, we define the DSM objective:

$$J_{\text{DSM}}(\theta) = \mathbb{E}_{p_{\tilde{\mathbf{x}},\mathbf{x}}}\left[\frac{1}{2}\left\|\mathbf{s}_\theta(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}}\log p_{\tilde{\mathbf{x}}|\mathbf{x}}(\tilde{\mathbf{x}}|\mathbf{x})\right\|^2\right] \tag{12}$$

The intuition is that the gradient of the log density at a corrupted point $\tilde{\mathbf{x}}$ should ideally move us toward the clean sample $\mathbf{x}$. With a Gaussian noise model, we have:

$$\nabla_{\tilde{\mathbf{x}}} \log p_{\tilde{\mathbf{x}}|\mathbf{x}}(\tilde{\mathbf{x}}|\mathbf{x}) = \frac{1}{\sigma^2}(\mathbf{x} - \tilde{\mathbf{x}}) \tag{13}$$

The direction $\frac{1}{\sigma^2}(\mathbf{x} - \tilde{\mathbf{x}})$ clearly represents the path from $\tilde{\mathbf{x}}$ back toward the clean sample $\mathbf{x}$, and we train our score model $\mathbf{s}_\theta$ to approximate this direction. This formulation bypasses the need to compute Hessians, making DSM computationally efficient.

3 Methods

To evaluate the effectiveness of various score estimation techniques, we conducted experiments on synthetic and real-world datasets. For synthetic experiments, we utilized a two-component Gaussian Mixture Model (GMM) with the following parameters:

$$\begin{aligned} \mu_1 &= 1.0, & \sigma_1 &= 0.5, & w_1 &= 0.3 \\ \mu_2 &= 6.0, & \sigma_2 &= 1.0, & w_2 &= 0.7 \end{aligned}$$

where μ_i, σ_i, and w_i represent the mean, standard deviation, and weight of each Gaussian component, respectively. This GMM configuration was chosen to create a clear bimodal distribution with sufficient separation between components to evaluate the score estimation accuracy in different density regions.

For real-world data experiments, we used the FashionMNIST dataset, which consists of 60,000 training and 10,000 test grayscale images of size 28×28 pixels, divided into 10 fashion product categories. The images were normalized to the range [-1, 1] for compatibility with our modeling approach. For training efficiency, we used a subset of 50,000 images from the training set.

3.0.1 Evaluation Methods

For the GMM experiments, we directly compared the estimated scores with the analytically calculated true scores across the input domain. This comparison provides a precise quantitative assessment of how accurately different methods can estimate the score function, particularly in low-density regions between the mixture components. For the FashionMNIST experiments, quantitative evaluation of score accuracy was not possible due to the lack of analytical score functions for complex real-world data. Instead, we assessed performance through visual inspection of generated samples.

3.0.2 Implementation Details

All experiments were implemented using PyTorch 1.9. The GMM experiments were conducted on a standard CPU machine, while the FashionMNIST experiments utilized a single NVIDIA Tesla T4 GPU with 16GB GDDR6 VRAM through Google Colab. For training, we used the Adam optimizer with a learning rate of 1e-3. Models were trained for 1000 epochs with a batch size of 128. For sampling in the reverse diffusion process, we used 100 timesteps for our experiments. We implemented two score estimation techniques for comparison: sliced score matching with a single random projection per sample and denoising score matching

with varying noise scales. This allowed us to assess the relative strengths and weaknesses of these approaches across different data settings.

3.1 Model Architecture

For the experiments in this paper, different neural network architectures were used to parameterize the score function depending on the complexity of the data distribution. For the GMM experiments, a simple multi-layer perceptron (MLP) was employed. The score network consisted of 4 hidden layers with 128 neurons each, using softplus activation functions. For the time-dependent score model, the network included a time embedding component through a separate branch that was combined with the main network using a concatenation operation.

For the FashionMNIST image experiments, a convolutional U-Net architecture was implemented to better handle the spatial structure of the data. The network consisted of an initial convolutional layer with 64 filters and kernel size 3×3, three convolutional blocks, each containing two convolutional layers with 64, 128, and 256 filters respectively, skip connections between corresponding layers and a final convolutional layer to map back to the image dimension. Softplus activation functions were used for each layer as well.

The time component was incorporated through a time embedding module that produced a 128-dimensional representation of the time variable. This embedding was then broadcast and added to each spatial location of the feature maps after the first convolutional layer, allowing the network to condition its score estimates on the specific noise level. All networks were implemented in PyTorch, and weights were initialized using the default PyTorch initialization for the respective layer types.

3.2 Training Procedure

For the selection of random projection vectors in the sliced score matching (SSM) approach. We sampled these vectors from a multivariate Rademacher distribution (uniform distribution over $\{-1, 1\}^D$) to ensure efficient computation of Hessian-vector products. For each data point in the batch, we generated a single random projection vector ($M = 1$), which we provided a good balance between computational efficiency and estimation accuracy [12]. The training procedure for each score estimation method was as follows:

Sliced Score Matching

For this method, we implemented Algorithm 1. Projecting the high-dimensional score function onto random directions, reduces computational complexity while preserving essential score information [12]. We used a batch size of 128 and a learning rate of 1e-3 for 1000 epochs, with early stopping based on validation loss to prevent overfitting.

Projection vectors were sampled from a multivariate Rademacher distribution (uniform over $\{-1, 1\}^D$). This choice offers two advantages: (1) it reduces variance in the estimator, as projection vectors maintain unit magnitude; and (2) it simplifies computation by avoiding floating-point operations with very small or large values that could lead to numerical instabilities. For each data point in the batch, we generated a single random projection vector

($M = 1$). While increasing M could theoretically improve estimation accuracy by averaging over more random projections, $M = 1$ was shown to provide a good balance between computational efficiency and estimation quality [12].

The objective function in Equation 11 was computed through a two-step process using automatic differentiation. First, we compute the score prediction $\mathbf{s}_m(\mathbf{x};\theta) = \nabla_{\mathbf{x}} \log \tilde{p}_m(\mathbf{x};\theta)$ through a single backward pass. Then, we compute the Hessian-vector product $\mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta)\mathbf{v}$ by taking the gradient of $\mathbf{v}^\top \mathbf{s}_m(\mathbf{x};\theta)$ with respect to $\mathbf{x}$. This approach requires only two gradient operations per sample, making it more efficient than traditional score matching, which requires D separate backward passes to compute the trace of the Hessian. The procedure for sliced score-matching is detailed in Algorithm 1:

Algorithm 1: Sliced Score Matching

Input: $\tilde{p}_m(\cdot;\theta)$, $\mathbf{x}$, $\mathbf{v}$
$\mathbf{s}_m(\mathbf{x};\theta) \leftarrow \text{grad}(\log \tilde{p}_m(\mathbf{x};\theta), \mathbf{x})$
$\mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta)\mathbf{v} \leftarrow \text{grad}(\mathbf{v}^\top \mathbf{s}_m(\mathbf{x};\theta), \mathbf{x})$
$J \leftarrow \frac{1}{2}(\mathbf{v}^\top \mathbf{s}_m(\mathbf{x};\theta))^2$
$J \leftarrow J + \mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta)\mathbf{v}$
return J

For the GMM experiments, we implemented a direct version of SSM that explicitly modeled the score function at different fixed noise levels. For the FashionMNIST experiments, we extended the algorithm to incorporate time conditioning, allowing a single model to learn score estimates across all noise scales simultaneously.

Denoising Score Matching

For the denoising score matching approach, we implemented Algorithm 2 to estimate the score function. Rather than directly estimating the score function, DSM trains a model to predict the direction that points from noisy data back toward clean data. We designed our noise schedule following [8] with $\sigma(t) = \sqrt{\frac{\exp(2t)-1}{2}}$, where $t \in [0, 1]$. This schedule increases exponentially with time, starting with very small noise at $t \approx 0$ and ending with substantial noise at $t \approx 1$.

During training, we sample time points uniformly from $\mathcal{U}(0, 1)$ for each batch element, rather than using fixed noise levels. This continuous-time approach ensures the model learns to denoise data across the entire spectrum of noise levels. The loss function can be interpreted as training the model to predict scaled noise vectors. Specifically, for a noisy sample $\mathbf{x}_t = \mathbf{x}_0 + \sigma(t) \cdot \mathbf{z}$, the score of the noisy distribution can be analytically derived as $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) = -\mathbf{z}/\sigma(t)$ under a Gaussian noise model. Our loss function directly trains the model to approximate this score:

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \| s_\theta(\mathbf{x}_t^{(i)}, t^{(i)}) \cdot \sigma(t^{(i)}) + \mathbf{z}^{(i)} \|^2$$

This loss formulation doesn't require computing Hessians or their traces, significantly reducing computational complexity. The complete algorithm for training with DSM is detailed below:

Algorithm 2: Denoising Score Matching Training

Require: Data distribution $p_d(\mathbf{x})$, score model $s_\theta(\mathbf{x}, t)$, noise schedule $\sigma(t) = \sqrt{\frac{\exp(2t)-1}{2} + 10^{-5}}$

Require: Learning rate α, number of epochs N, batch size B

Initialize model parameters θ randomly
for epoch $= 1$ to N **do**
  for each batch $\{\mathbf{x}_0^{(i)}\}_{i=1}^{B} \sim p_d(\mathbf{x})$ **do**
    Sample random time points $t^{(i)} \sim \mathcal{U}(0, 1 - 10^{-5})$ for $i = 1$ to B
    Compute noise std $\sigma(t^{(i)})$ for each sample
    Sample noise $\mathbf{z}^{(i)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $i = 1$ to B
    Create noisy samples $\mathbf{x}_t^{(i)} = \mathbf{x}_0^{(i)} + \sigma(t^{(i)}) \cdot \mathbf{z}^{(i)}$
    Compute score predictions $s_\theta(\mathbf{x}_t^{(i)}, t^{(i)})$
    Compute loss $\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \| s_\theta(\mathbf{x}_t^{(i)}, t^{(i)}) \cdot \sigma(t^{(i)}) + \mathbf{z}^{(i)} \|^2$
    Update parameters $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$
  end for
  Evaluate on validation set (10% of training data) and save best model
end for
return Trained model parameters θ

3.3 Forward SDE

For our experiments, we utilized an SDE defined as:

$$d\mathbf{x} = e^t d\mathbf{w} \tag{14}$$

where $t \in [0, 1]$ and the diffusion coefficient $g(t) = e^t$ increases exponentially with time. This SDE has no drift term ($\mathbf{f}(\mathbf{x}, t) = \mathbf{0}$) and the noise variance at time t is given by:

$$\sigma^2(t) = \frac{e^{2t} - 1}{2} \tag{15}$$

The forward process gradually transforms data points from the original distribution p_0 to a noise distribution. Given a data point $\mathbf{x}_0 \sim p_0$, the perturbed sample at time t can be directly computed as:

$$\mathbf{x}_t = \mathbf{x}_0 + \sqrt{\sigma^2(t)} \cdot \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{16}$$

This leads to a time-dependent marginal distribution $p_t(\mathbf{x})$ whose score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ we aim to estimate with our neural network $s_\theta(\mathbf{x}, t)$.

3.4 Reverse SDE

To generate samples, we employed the reverse-time SDE:

$$d\mathbf{x} = -e^{2t} s_\theta(\mathbf{x}, t) dt + e^t d\bar{\mathbf{w}} \tag{17}$$

This is derived from the general form of the reverse SDE (Equation 8) by substituting our specific diffusion coefficient $g(t) = e^t$ and zero drift. Numerical integration of this reverse

SDE was performed using the Euler-Maruyama method, with a discretization of 100 time steps with following procedure:

$$\begin{aligned}\Delta\mathbf{x} &\leftarrow [-e^{2t}s_\theta(\mathbf{x}, t)dt]\Delta t + e^t\sqrt{|\Delta t|}\mathbf{z}_t \\ \mathbf{x} &\leftarrow \mathbf{x} + \Delta\mathbf{x} \\ t &\leftarrow t + \Delta t\end{aligned}$$

where $\mathbf{z}_t \sim \mathcal{N}(0, I)$. Starting with a sample from the prior distribution $\mathbf{x}_1 \sim \mathcal{N}(\mathbf{0}, \sigma^2(1)\mathbf{I})$, we iteratively solved the reverse SDE from $t = 1$ to $t = 0$. The final sample $\mathbf{x}_0$ represents our generated data point.

4 Results

In this section, we present the experimental results of our score-based generative models using different score estimation techniques. We first examine the performance on the synthetic Gaussian Mixture Model (GMM) task, where accurate evaluation is possible through comparison with true analytical scores. We then demonstrate the application of these methods to the more complex FashionMNIST dataset, assessing the quality of generated samples through visual inspection.

4.1 Gaussian Mixture Model

For the GMM experiments, we evaluated how accurately different methods can approximate the true score function. Figures 3 and 4 show a comparison between the true scores of our two-component GMM and the approximated scores obtained via sliced score matching (SSM) and denoising score matching (DSM) at three different time points ($t = 0.1$, $t = 0.5$, and $t = 0.9$).

Our results indicate that both methods provide reasonable approximations of the true score function, but with notable differences in performance. The SSM approach (Figure 3) demonstrates more consistent score estimation across different regions of the distribution, particularly maintaining accuracy in the critical transition areas between the two Gaussian components. The approximated scores closely track the true scores at each time point, with only minor deviations observed at the distribution tails.

In contrast, the DSM approach (Figure 4) shows more pronounced differences between the true and approximated scores, especially at the earlier time point ($t = 0.1$). The approximated score function exhibits smoother transitions but fails to capture the sharp changes in the true score, particularly at the boundaries between high and low density regions. At later time points, as the distribution becomes more diffused, the DSM approximation improves but still demonstrates less accuracy compared to SSM.

Despite these differences in score estimation accuracy, both methods successfully recover the target distribution as shown in Figures 5a and 5b. The SSM-generated samples (Figure 5a) closely match the original bimodal distribution, accurately capturing both the relative heights and positions of the two modes. However, we observe a slight overestimation of the density in the rightmost mode.

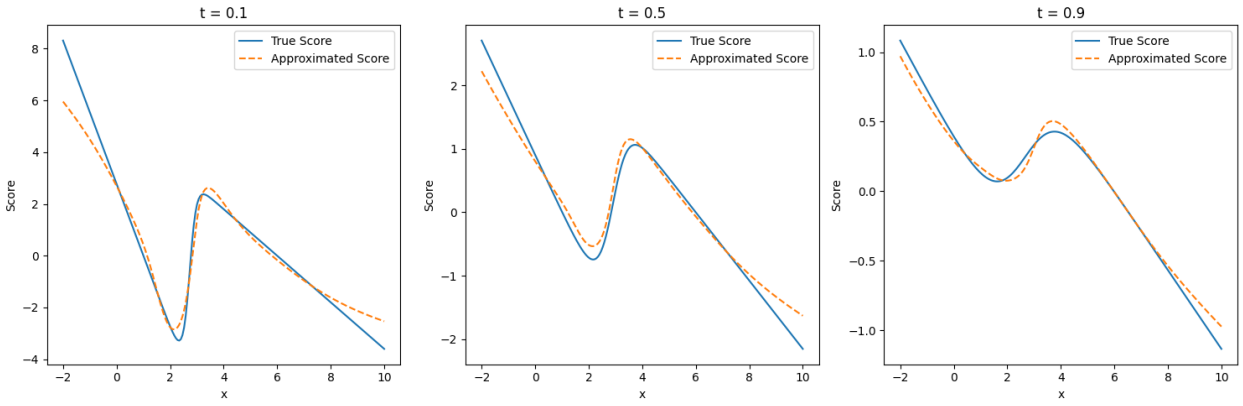


Figure 3: Comparison of true score (solid blue) vs sliced score estimate (dashed orange) of the gaussian mixture model across three time points.

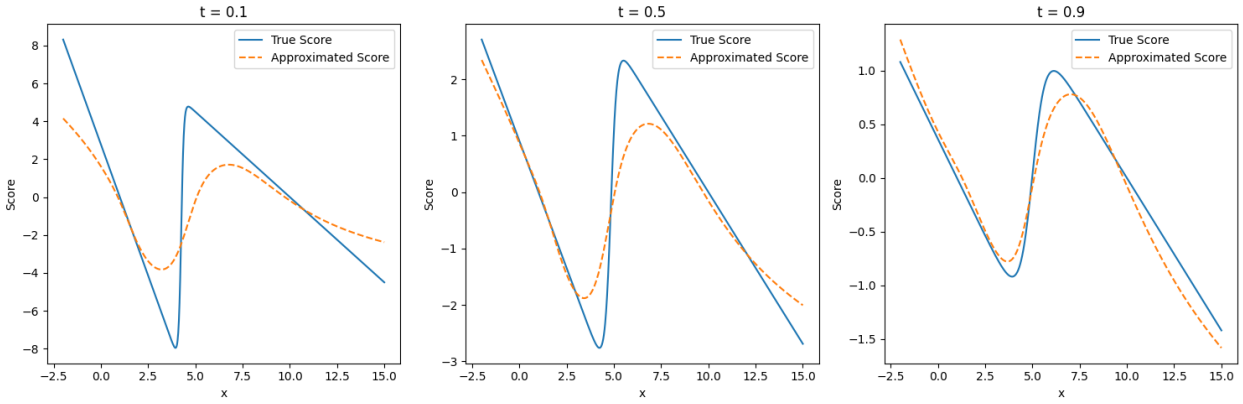


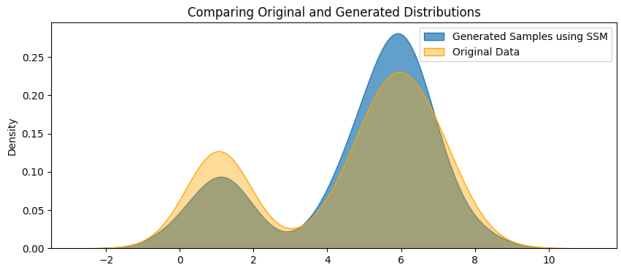
Figure 4: Comparison of true score (solid blue) vs denoised score estimate (dashed orange) of the gaussian mixture model across three time points.

The DSM approach (Figure 5b) also produces a distribution that generally follows the target GMM, but with some discrepancies. The left mode is reconstructed with high fidelity, but the right mode shows a more pronounced deviation in both height and spread compared to the ground truth. This aligns with our observation that DSM's score estimation is less accurate in certain regions of the distribution.

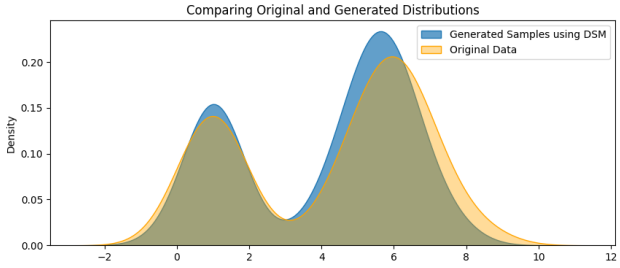
4.2 FashionMNIST

For the more complex FashionMNIST dataset, we evaluated both SSM and DSM approaches on their ability to generate realistic fashion item images. Figures 6b & 6a presents a side-by-side comparison of samples generated using both methods.

The visual quality of the generated samples reveals significant differences between the two approaches. The DSM-generated images (Figure 6a) show recognizable fashion items but suffer from noticeable artifacts, inconsistent structures, and less defined boundaries. Many items appear blurry or incomplete, with some samples displaying unnatural distortions that make it difficult to identify the specific clothing category.



(a) Comparison of target Gaussian mixture distribution (orange) vs generated samples (blue) from using sliced score matching.



(b) Comparison of target Gaussian mixture distribution (orange) vs generated samples (blue) from using denoised score matching.

(a) FashionMNIST sample generation using DSM

(b) FashionMNIST Sample generation using SSM

In contrast, the SSM-generated images (Figure6b exhibit substantially higher quality with clearer outlines, more consistent structures, and better preservation of characteristic features that define each clothing category. The samples demonstrate greater visual fidelity to the original FashionMNIST dataset, with recognizable shirts, pants, shoes, and dresses that maintain appropriate proportions and detail levels.

5 Discussion

Our experimental results demonstrate that both sliced score matching (SSM) and denoising score matching (DSM) enable effective generative modeling through score estimation, but with distinct strengths and limitations that become apparent across different data complexity levels.

5.1 Accuracy of Score Estimation

For the one-dimensional GMM task, SSM provided more accurate score approximations compared to DSM, particularly in the critical low-density regions between mixture components. This aligns with theoretical expectations, as SSM's random projection approach partially mitigates the density weighting inherent in traditional score matching objectives. By projecting scores onto random directions, SSM captures directional information more uniformly across the entire domain, leading to better performance in regions where data is sparse.

DSM, while computationally efficient, demonstrated less accuracy in low-density regions, particularly at early time points in the diffusion process. This limitation stems from its training objective, which still inherits the fundamental challenge of score matching—errors in regions of low data density receive less weight in the overall loss function. Since DSM is trained to denoise samples back toward high-density regions, it has reduced incentive to learn accurate scores in areas rarely visited during training.

Interestingly, despite these differences in score estimation accuracy, both methods successfully recovered the target distribution structure. This suggests that generative performance may be robust to certain types of score approximation errors, particularly when the errors occur in regions that have minimal impact on the overall sampling procedure. This trend was also exhibited against the performance on the FashionMNIST dataset. Generated images produced via sliced score matching were of better quality compared to images produced from denoised score matching. One aspect to note is that sliced score matching was also slower to compute than denoised score matching for the same number of training episodes. SSM offers computational advantages over traditional score matching by avoiding explicit computation of the score function's Hessian trace, instead using random projections to estimate the necessary terms. DSM, while requiring paired clean and noisy samples, offers a straightforward objective function that avoids second derivatives entirely.

5.2 Limitations

While our study provides valuable insights into score estimation techniques, several limitations should be acknowledged. Our experiments focused on relatively simple datasets; testing on more complex, high-resolution images would further validate the dimensional scaling patterns we observed. We explored only two score estimation methods, leaving other promising approaches like flow-based score matching unexplored. There are more elaborate neural network architectures that can also be employed which have shown to produce better results with denoised score matching and improve computational efficiency.

Song et al. [8] also defined three different hand-designed SDEs, specifically variance exploding, variance preserving and sub-variance preserving SDEs, that they recommend when implementing generating diffusion models. We were not able to investigate these or other SDEs in this paper. The sub-par results from DSM can be attributed to the SDE and hyperparameter σ selection in Equation 13. DSM performance has been shown to be sensitive to the appropriate value of σ [14].

5.3 Connection to Flow Matching

An important theoretical direction for future work is exploring the relationship between score-based models and flow matching. Flow matching represents a generalization of the diffusion framework, where instead of learning the score function, one directly learns a vector field that transforms samples from a simple distribution to the data distribution [15].

Score-based diffusion models can be viewed as a special case of flow matching when using Gaussian probability paths. In this context, the score function and vector field are directly related, and either can be used to define the generative process. While score matching focuses on learning gradients of log density, flow matching learns the transformation path directly. This perspective unifies several generative approaches under a common theoretical framework. It's also been shown that using a time-reversal is not necessary and often leads to suboptimal results [16]. Future research could leverage this connection to develop hybrid methods that combine the computational efficiency of score matching with the flexibility of flow matching. For example, models could adaptively switch between score-based and flow-based objectives depending on the complexity of different regions in the data space.

Score-based generative models represent a powerful framework that combines the strengths of both explicit density modeling and implicit sampling-based approaches. By directly estimating the score function, these models avoid many challenges associated with density normalization while maintaining the ability to explicitly represent probability gradients. As demonstrated in the experiments outlined in this paper, advances in score estimation techniques continue to push the boundaries of what's possible in generative modeling.

References

[1] Yan Liu and He Wang. *Who on Earth Is Using Generative AI?* Policy Research working paper. Washington, D.C.: World Bank Group, 2023. URL: http://documents.worldbank.org/curated/en/099720008192430535.

[2] L. Metz et al. "Unrolled Generative Adversarial Networks". In: *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings* (2017). URL: https://openreview.net/forum?id=BydrOIcle.

[3] Tim Salimans et al. "Improved Techniques for Training GANs". In: *Advances in Neural Information Processing Systems* (2016), pp. 2226–2234.

[4] Hugo Larochelle and Iain Murray. "The Neural Autoregressive Distribution Estimator". In: *International Conference on Artificial Intelligence and Statistics* (2011), pp. 29–37.

[5] Diederik P Kingma and Max Welling. "Auto-Encoding Variational Bayes". In: *International Conference on Learning Representations* (2014).

[6] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. "Density Estimation using Real NVP". In: *International Conference on Learning Representations* (2017).

[7] Yang Song and Stefano Ermon. "Generative Modeling by Estimating Gradients of the Data Distribution". In: *Advances in Neural Information Processing Systems* (2019), pp. 11895–11907.

[8] Yang Song et al. "Score-Based Generative Modeling through Stochastic Differential Equations". In: *International Conference on Learning Representations* (2021).

[9] Aapo Hyvarinen. "Estimation of Non-Normalized Statistical Models by Score Matching". In: *Journal of Machine Learning Research* 6 (2005), pp. 695–709.

[10] G. Parisi. "Correlation functions and computer simulations". In: *Nuclear Physics B* 180.3 (1981), pp. 378–384. ISSN: 0550-3213. DOI: https://doi.org/10.1016/0550-3213(81)90056-0. URL: https://www.sciencedirect.com/science/article/pii/0550321381900560.

[11] B. D. Anderson. "Reverse-Time Diffusion Equation Models". In: *Stochastic Processes and their Applications* 12.3 (1982), pp. 313–326.

[12] Yang Song et al. *Sliced Score Matching: A Scalable Approach to Density and Score Estimation*. en. arXiv:1905.07088 [cs]. June 2019. DOI: 10.48550/arXiv.1905.07088. URL: http://arxiv.org/abs/1905.07088 (visited on 02/12/2025).

[13] Pascal Vincent. "A Connection Between Score Matching and Denoising Autoencoders". In: *Neural Computation* 23.7 (2011), pp. 1661–1674.

[14] Yang Song and Stefano Ermon. "Improved Techniques for Training Score-Based Generative Models". In: *Advances in Neural Information Processing Systems 33* (2020).

[15] Yaron Lipman et al. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: https://arxiv.org/abs/2210.02747.

[16] Samuel Lavoie et al. *Modeling Caption Diversity in Contrastive Vision-Language Pretraining*. 2025. arXiv: 2405.00740 [cs.CV]. URL: https://arxiv.org/abs/2405.00740.

A Score Function Independence from Partition Function

The score function is independent of the partition function as shown by the following:

$$\begin{aligned}
\mathbf{s}_m(\mathbf{x};\theta) &= \nabla_{\mathbf{x}} \log p_m(\mathbf{x};\theta) \\
&= \nabla_{\mathbf{x}} \log \frac{\tilde{p}_m(\mathbf{x};\theta)}{Z_\theta} \\
&= \nabla_{\mathbf{x}} \left[\log \tilde{p}_m(\mathbf{x};\theta) - \log Z_\theta\right] \\
&= \nabla_{\mathbf{x}} \log \tilde{p}_m(\mathbf{x};\theta) - \nabla_{\mathbf{x}} \log Z_\theta \\
&= \nabla_{\mathbf{x}} \log \tilde{p}_m(\mathbf{x};\theta)
\end{aligned}$$

B Derivation of Score Matching Objective $J(\theta)$

We begin with the Fisher divergence between the data distribution p_d and model distribution $p_m(\cdot;\theta)$:

$$L(\theta) \triangleq \frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_m(\mathbf{x};\theta) - \mathbf{s}_d(\mathbf{x})||_2^2]$$

where $\mathbf{s}_m(\mathbf{x};\theta) \triangleq \nabla_{\mathbf{x}} \log p_m(\mathbf{x};\theta)$ and $\mathbf{s}_d(\mathbf{x}) \triangleq \nabla_{\mathbf{x}} \log p_d(\mathbf{x})$ are the score functions.

Let's expand the squared ℓ_2 norm:

$$\begin{aligned}
L(\theta) &= \frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_m(\mathbf{x};\theta) - \mathbf{s}_d(\mathbf{x})||_2^2] \\
&= \frac{1}{2}\mathbb{E}_{p_d}[\sum_{i=1}^{D}([\mathbf{s}_m(\mathbf{x};\theta)]_i - [\mathbf{s}_d(\mathbf{x})]_i)^2] \\
&= \frac{1}{2}\mathbb{E}_{p_d}[\sum_{i=1}^{D}([\mathbf{s}_m(\mathbf{x};\theta)]_i^2 - 2[\mathbf{s}_m(\mathbf{x};\theta)]_i[\mathbf{s}_d(\mathbf{x})]_i + [\mathbf{s}_d(\mathbf{x})]_i^2)] \\
&= \frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_m(\mathbf{x};\theta)||_2^2] - \mathbb{E}_{p_d}[\mathbf{s}_m(\mathbf{x};\theta)^T\mathbf{s}_d(\mathbf{x})] + \frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_d(\mathbf{x})||_2^2]
\end{aligned}$$

The third term $\frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_d(\mathbf{x})||_2^2]$ is a constant with respect to θ, so we can represent it as the constant C in $L(\theta) = J(\theta) + C$.

The second term:

$$-\mathbb{E}_{p_d}[\mathbf{s}_m(\mathbf{x};\theta)^T\mathbf{s}_d(\mathbf{x})] = -\mathbb{E}_{p_d}\left[\sum_{i=1}^{D}[\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot [\mathbf{s}_d(\mathbf{x})]_i\right]$$

For each dimension i, we have:

$$\begin{aligned}
-\mathbb{E}_{p_d}[[\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot [\mathbf{s}_d(\mathbf{x})]_i] &= -\mathbb{E}_{p_d}\left[[\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot \frac{\partial \log p_d(\mathbf{x})}{\partial x_i}\right] \\
&= -\mathbb{E}_{p_d}\left[[\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot \frac{1}{p_d(\mathbf{x})}\frac{\partial p_d(\mathbf{x})}{\partial x_i}\right] \\
&= -\int [\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot \frac{\partial p_d(\mathbf{x})}{\partial x_i} d\mathbf{x}
\end{aligned}$$

Now, we use integration by parts, where:

$$\begin{aligned}
u &= [\mathbf{s}_m(\mathbf{x};\theta)]_i \\
dv &= \frac{\partial p_d(\mathbf{x})}{\partial x_i} d\mathbf{x}
\end{aligned}$$

This gives:

$$-\int [\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot \frac{\partial p_d(\mathbf{x})}{\partial x_i} d\mathbf{x} = -\left.[\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot p_d(\mathbf{x})\right|_{-\infty}^{\infty} + \int \frac{\partial [\mathbf{s}_m(\mathbf{x};\theta)]_i}{\partial x_i} \cdot p_d(\mathbf{x}) d\mathbf{x}$$

Assuming that $p_d(\mathbf{x})$ decreases faster than $[\mathbf{s}_m(\mathbf{x};\theta)]_i$ increases as $||\mathbf{x}|| \to \infty$, the boundary term vanishes, giving:

$$\begin{aligned}
-\int [\mathbf{s}_m(\mathbf{x};\theta)]_i \cdot \frac{\partial p_d(\mathbf{x})}{\partial x_i} d\mathbf{x} &= \int \frac{\partial [\mathbf{s}_m(\mathbf{x};\theta)]_i}{\partial x_i} \cdot p_d(\mathbf{x}) d\mathbf{x} \\
&= \mathbb{E}_{p_d}\left[\frac{\partial [\mathbf{s}_m(\mathbf{x};\theta)]_i}{\partial x_i}\right]
\end{aligned}$$

Applying this to all dimensions:

$$\begin{aligned}
-\mathbb{E}_{p_d}[\mathbf{s}_m(\mathbf{x};\theta)^T \mathbf{s}_d(\mathbf{x})] &= \mathbb{E}_{p_d}\left[\sum_{i=1}^{D} \frac{\partial [\mathbf{s}_m(\mathbf{x};\theta)]_i}{\partial x_i}\right] \\
&= \mathbb{E}_{p_d}[\mathrm{tr}(\nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta))]
\end{aligned}$$

where $\mathrm{tr}(\nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta))$ is the trace of the Hessian of $\log p_m(\mathbf{x};\theta)$.
Substituting back into our expanded expression for $L(\theta)$:

$$L(\theta) = \frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_m(\mathbf{x};\theta)||_2^2] + \mathbb{E}_{p_d}[\mathrm{tr}(\nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta))] + \frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_d(\mathbf{x})||_2^2]$$

The term $\frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_d(\mathbf{x})||_2^2]$ is constant with respect to θ, so:

$$L(\theta) = \underbrace{\mathbb{E}_{p_d}\left[\mathrm{tr}(\nabla_{\mathbf{x}} \mathbf{s}_m(\mathbf{x};\theta)) + \frac{1}{2}||\mathbf{s}_m(\mathbf{x};\theta)||_2^2\right]}_{J(\theta)} + \underbrace{\frac{1}{2}\mathbb{E}_{p_d}[||\mathbf{s}_d(\mathbf{x})||_2^2]}_{C}$$

Therefore:

$$J(\theta) \triangleq \mathbb{E}_{p_d}\left[\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_m(\mathbf{x};\theta)) + \frac{1}{2}||\mathbf{s}_m(\mathbf{x};\theta)||_2^2\right]$$

This is the score matching objective function that can be minimized without requiring knowledge of the true score function $\mathbf{s}_d(\mathbf{x})$.